Requirements Analysis Document

Autonomous AI Job Orchestrator

Version: 1.0
Date: February 15, 2026
Authors: J. A. L. Perera (AS2022939), A. A. Lokuliyana (AS2022921)

---

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the **Autonomous AI Job Orchestrator**. This system is designed to replace traditional static job schedulers (FIFO, Round-Robin) with an intelligent, self-optimizing engine powered by Reinforcement Learning (Deep Q-Networks). The system focuses on handling autonomous workloads with distinct priorities, deadlines, and dependencies.

1.2 Scope

The system acts as a central control plane for distributed job execution. It includes:

- A REST API for job submission and management.
- A Redis-backed state store for persistence.
- An AI-driven Scheduler that prioritizes jobs based on learned rewards.
- A distributed Worker pool for asynchronous execution.
- A Monitoring Dashboard for real-time observability.
- Mechanisms for DAG (Directed Acyclic Graph) dependency resolution and failure recovery.

1.3 Definitions, Acronyms, and Abbreviations

- **DAG**: Directed Acyclic Graph (represents job dependencies).
- **RL**: Reinforcement Learning.
- **DQN**: Deep Q-Network.
- **FIFO**: First-In-First-Out.
- **API**: Application Programming Interface.

- **Zombie Job**: A job that remains in a 'RUNNING' state indefinitely due to worker failure.

---

2. Overall Description

2.1 Product Perspective

This software is a standalone orchestration engine intended to run in a containerized environment (Docker). It interfaces with clients via HTTP/REST and allows for horizontal scaling of worker nodes to handle increased load.

2.2 User Characteristics

- **System Administrators**: manage the infrastructure, scale workers, and monitor system health via the dashboard.
- **Developers/Applications**: Submit jobs programmatically via the REST API.
- **Data Scientists/Analysts**: Observe the behavior of the RL Agent and optimize reward functions.

2.3 Assumptions

- The system runs on a network supporting HTTP and TCP (for Redis).
- Jobs are simulation-based (sleep for duration) for this prototype but represent real-world compute tasks.
- Redis is available as the primary persistence layer.

---

3. Functional Requirements

3.1 Job Submission & Management

- **FR-01**: The system must provide a REST API endpoint (`POST /api/v1/jobs/`) to accept new job submissions.
- **FR-02**: Jobs must accept parameters including `name` , `priority` (1-10), `estimated_duration` , `deadline` , and `dependencies` .
- **FR-03**: The system must generate a unique UUID for every submitted job.
- **FR-04**: The system must allow users to retrieve the status of a specific job (`GET /api/v1/jobs/{id}`).

3.2 Scheduling Logic

- **FR-05**: The Scheduler must periodically poll for `PENDING` jobs.
- **FR-06 (DAG)**: A job should only be considered "runnable" if all its dependencies (parent jobs) are in the `COMPLETED` state.
- **FR-07 (AI)**: The system must use a Reinforcement Learning Agent to select the best job from the "runnable" pool based on the current state vector (Priority, Deadline, Duration).
- **FR-08**: The system must support a fallback mechanism (FIFO) if the AI makes invalid selections during exploration.

3.3 Job Execution (Distributed Workers)

- **FR-09**: The Scheduler must push selected job IDs to a Redis Queue.
- **FR-10**: Distributed Worker processes must pop jobs from the queue and execute them asynchronously.
- **FR-11**: Workers must update job status to `RUNNING` upon start and `COMPLETED` upon finish.
- **FR-12**: Workers must handle execution errors and mark jobs as `FAILED`.

3.4 Self-Optimization (Machine Learning)

- **FR-13**: The system must calculate a "Reward" score after every job completion based on:
  - Meeting the Deadline (+Reward).
  - Job Priority (+Reward).
  - Failure or Delay (-Penalty).
- **FR-14**: The RL Agent must train (update neural weights) using the observed State, Action, and Reward.
- **FR-15**: The system must persist the trained model (`ai_brain.pth`) to disk to retain learning across restarts.

3.5 Fault Tolerance & Self-Healing

- **FR-16**: The Scheduler must include a "Zombie Killer" monitor.
- **FR-17**: Any job running longer than a defined threshold (`JOB_TIMEOUT_SECONDS`) must be automatically marked as `FAILED`.

3.6 Monitoring Dashboard

- **FR-18**: The system must provide a web-based dashboard (Streamlit).
- **FR-19**: The dashboard must display real-time metrics: Pending count, Running count, Completed count.
- **FR-20**: The dashboard must visualize the active job queue sorted by priority.

---

4. Non-Functional Requirements

4.1 Scalability

- **NFR-01**: The system must support horizontal scaling of Worker nodes without modifying the Scheduler code.
- **NFR-02**: The system must be containerized (Docker) to allow easy deployment of multiple instances.

4.2 Performance

- **NFR-03**: The API must respond to job submission requests within 200ms under normal load.
- **NFR-04**: The Scheduler loop must process pending jobs at least every 2 seconds.

4.3 Reliability

- **NFR-05**: The system state must be persisted in Redis to survive application restarts.
- **NFR-06**: The Scheduler must not crash if the AI Agent predicts an invalid action; it must recover gracefully.

4.4 Maintainability

- **NFR-07**: The codebase must follow modular design principles (separation of concerns: API, Scheduler, Worker, RL Engine).
- **NFR-08**: Comprehensive documentation and unit tests must be provided.

---

5. Interface Requirements

5.1 System Interfaces

- **REST API**: JSON over HTTP.
- **Database**: Redis Protocol (RESP) on port 6379.

5.2 User Interface

- **Command Line**: For starting components and running test scripts.
- **Web Dashboard**: Streamlit interface accessible via browser on port 8501.

---

6. Data Requirements

6.1 Job Data Model

The system acts upon a `Job` entity containing:

Field	Type	Description
`id`	UUID	Unique Identifier
`name`	String	Human-readable name
`status`	Enum	PENDING, RUNNING, COMPLETED, FAILED
`priority`	Integer	Level 1 (Low) to 10 (High)
`created_at`	DateTime	Timestamp of submission
`started_at`	DateTime	Timestamp of execution start
`completed_at`	DateTime	Timestamp of completion
`dependencies`	List[UUID]	IDs of prerequisite jobs

---

7. Constraints

- **C-01**: Implementation language is Python 3.10+.
- **C-02**: Must use PyTorch for the RL component.
- **C-03**: Must run on standard consumer hardware for demonstration purposes.